

Quality Assurance Vs Quality Control: What's the Difference?

What is Quality Assurance?

Quality Assurance is popularly known as QA Testing, is defined as an activity to ensure that an organization is providing the best possible product or service to customers.

What is Quality Control?

Quality control popularly abbreviated as QC. It is a Software Engineering process used to ensure quality in a product or a service. It does not deal with the processes used to create a product; rather it examines the quality of the "end products" and the final outcome.

This tutorial gives the difference between [Quality Assurance](#) and Quality Control

Difference between Quality Assurance (QA) and Quality Control (QC)

Quality Assurance (QA)	Quality Control (QC)
It is a procedure that focuses on providing assurance that quality requested will be achieved.	It is a procedure that focuses on fulfilling the quality requested.
QA aims to prevent the defect	QC aims to identify and fix defects
It is a method to manage the quality- Verification	It is a method to verify the quality-Validation

• It does not involve executing the program	• It always involves executing a program
• It's a Preventive technique	• It's a Corrective technique
• It's a Proactive measure	• It's a Reactive measure
• It is the procedure to create the deliverables	• It is the procedure to verify that deliverables
• QA involves in full software development life cycle	• QC involves in full software testing life cycle
• In order to meet the customer requirements, QA defines standards and methodologies	• QC confirms that the standards are followed while working on the product
• It is performed before Quality Control	• It is performed only after QA activity is done
• It is a Low-Level Activity, it can identify an error and mistakes which QC cannot	• It is a High-Level Activity, it can identify an error that QA cannot
• Its main motive is to prevent defects in the system. It is a less time-consuming activity	• Its main motive is to identify defects or bugs in the system. It is a more time-consuming activity
• QA ensures that everything is executed in the right way, and that is why it falls under	• QC ensures that whatever we have done is as per the requirement, and that is why it

verification activity	falls under validation activity
• It requires the involvement of the whole team	• It requires the involvement of the Testing team
• The statistical technique applied on QA is known as SPC or Statistical Process Control (SPC)	• The statistical technique applied to QC is known as SQC or Statistical Quality Control

KEY DIFFERENCE

- Quality Assurance is aimed to avoid the defect whereas Quality control is aimed to identify and fix the defects.
- Quality Assurance provides assurance that quality requested will be achieved whereas Quality Control is a procedure that focuses on fulfilling the quality requested.
- Quality Assurance is done in software development life cycle whereas Quality Control is done in software testing life cycle.
- Quality Assurance is a proactive measure whereas Quality Control is a Reactive measure.
- Quality Assurance requires the involvement of all team members whereas Quality Control needs only testing team.
- Quality Assurance is performed before Quality Control.

Difference Between Verification and Validation with Example

What is Verification?

The verifying process includes checking documents, design, code, and program.

What is Validation?

Validation is a dynamic mechanism of Software testing and validates the actual product.

Verification vs Validation: Key Difference

Verification	Validation
The verifying process includes checking documents, design, code, and program	It is a dynamic mechanism of testing and validating the actual product
It does <i>not</i> involve executing the code	It always involves executing the code
Verification uses methods like reviews, walkthroughs, inspections, and desk- checking etc.	It uses methods like Black Box Testing, White Box Testing, and non-functional testing
Whether the software conforms to specification is checked	It checks whether the software meets the requirements and expectations of a customer
It finds bugs early in the development cycle	It can find bugs that the verification process can not catch
Target is application and software architecture, specification, complete design, high level, and database design etc.	Target is an actual product
QA team does verification and make sure that the software is as per the requirement in the SRS	With the involvement of testing team validation is executed on

document.	software code.
• It comes before validation	• It comes after verification

Example of verification and validation

- In Software Engineering, consider the following specification

A clickable button with name Submet

- Verification would check the design doc and correcting the spelling mistake.
- Otherwise, the development team will create a button like



- So new specification is ***A clickable button with name Submit***
- Once the code is ready, Validation is done. A Validation test found –



- Owing to Validation testing, the development team will make the submit button clickable

Positive Testing and Negative Testing with Examples

Software testing is the process of verifying and validating a software application to check whether it is working as expected. The intent is to find defects and improve product quality. There are two ways to test software, namely Positive Testing, and Negative Testing.

What is Positive Testing?

Positive testing is the type of testing that can be performed on the system by providing the **valid data as input**. It checks whether an application behaves as expected with positive inputs. This test is done to check the application that does what it is supposed to do.

For example -

Enter Only Numbers

Positive Testing

There is a text box in an application which can accept only numbers. Entering values up to 99999 will be acceptable by the system and any other values apart from this should not be acceptable. To do positive testing, set the valid input values from 0 to 99999 and check whether the system is accepting the values.

What is Negative Testing?

Negative Testing is a variant of testing that can be performed on the system by providing **invalid data as input**. It checks whether an application behaves as expected with the negative inputs. This is to test the application does not do anything that it is not supposed to do so.

For example -

Enter Only Numbers

Negative Testing

Negative testing can be performed by entering characters A to Z or from a to z. Either software system should not accept the values or else it should throw an error message for these invalid data inputs.

In both the testing, the following needs to be considered:

- Input data
- An action which needs to be performed
- Output Result

Testing Technique used for Positive and Negative Testing:

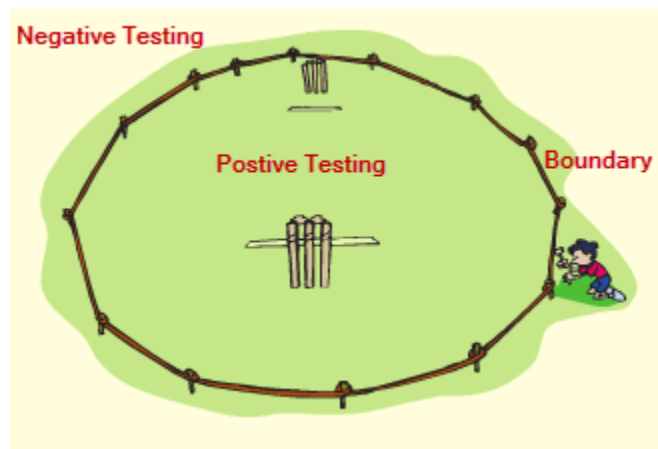
Following techniques are used for Positive and negative validation of testing is:

- Boundary Value Analysis
- Equivalence Partitioning

Boundary Value Analysis:

This is one of the software testing technique in which the test cases are designed to include values at the boundary. If the input data is used within

the boundary value limits, then it is said to be Positive Testing. If the input data is picked outside the boundary value limits, then it is said to be Negative Testing.

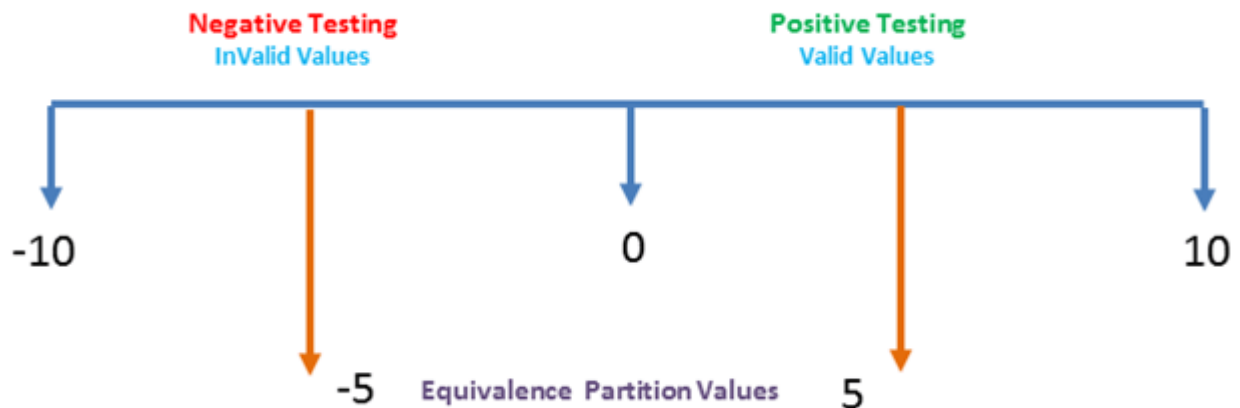


For example -

A system can accept the numbers from 0 to 10 numeric values. All other numbers are invalid values. Under this technique, boundary values -1,0,1 and 9,10,11 will be tested.

Equivalence Partitioning:

This is a software testing technique which divides the input data into many partitions. Values from each partition must be tested at least once. Partitions with valid values are used for Positive Testing. While partitions with invalid values are used for negative testing.



For example-

Numeric values Zero to ten can be divided into two (or three) partitions. In our case, we have two partitions -10 to -1 and 0 to 10. Sample values (5 and -5) can be taken from each part to test the scenarios.

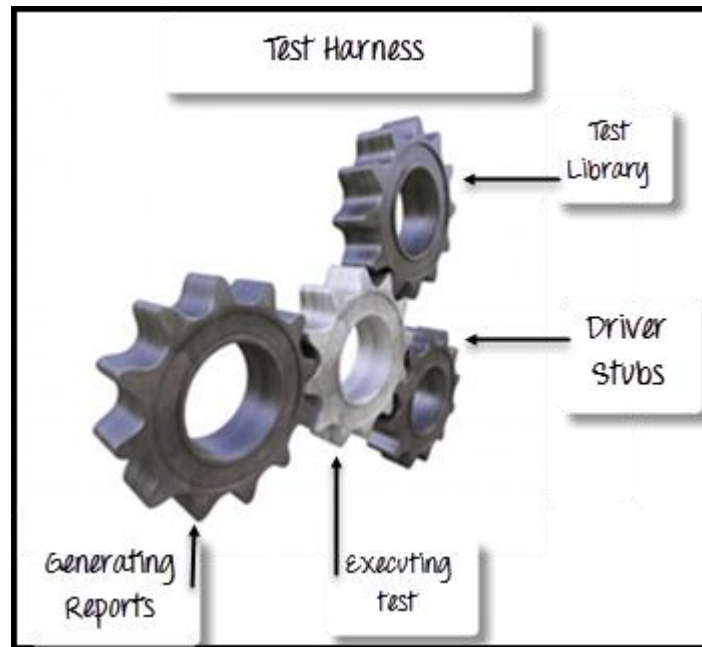
Conclusion:

Testing helps deliver quality software application and ensures the software is bug-free before the software is launched. For effective testing, use both - Positive and Negative testing which give enough confidence in the quality of the software. Real-time users can input any values and those needs to be tested before release.

What is TEST HARNESS? Tools & Examples

What is a Test Harness?

TEST HARNESS is a collection of stubs, drivers, and other supporting tools that are required to execute tests. Test harness executes tests, by using a test library and generates a report. Test harness contains all the information needed to compile and run a test like test cases, target deployment port(TDP), source file under test, stubs, etc.



Why use Test Harness?

- Automate the testing process
- Execute test suites of test cases
- Generate associated test reports
- Support for debugging
- To record the test results for each one of the tests
- Helps the developers to measure code coverage at a code level
- Increase the productivity of the system through automation
- Enhance the quality of software components and application
- To handle the complex condition that testers are finding difficult to simulate

There are two contexts where Test Harness is used

1. **Automation testing:** It contains the test scripts, parameters necessary to run these scripts and gather results to analyze it
2. **Integration testing:** It is used to put together two units of code or module that interact with each other to check whether or not the combined behavior is as expected or not

Test Harness Tools

- [JUnit](#): Tool used while using [Java](#)
- [Nunit](#): Tool used for using .Net framework

Test Harness Vs Test Framework

Test Harness	Test Automation Framework
• A test harness is composed of drivers and stubs, which are small dummy programs that interact with the software under test	• It is a set of processes, procedures, abstract concept and an environment in which automated tests are designed and implemented
• You can not "Record & Playback" script in Test Harness	• A tester can manually "Record & Playback" script in this framework
• Test harness contains all the information needed to compile and run a test like test cases, target deployment port(TDP), source file under test, stubs, etc.	• Test automation framework contains information like test library, testing tools, automated testing practices, a testing platform, etc.
• A test harness is categorized into ◦ Automation Testing ◦ Integration Testing	• Automation framework examples ◦ Data-driven testing ◦ Keyword driven testing ◦ Modularity driven testing ◦ Hybrid testing ◦ Model-based testing ◦ Code driven testing ◦ Behavior-driven testing

Test Condition vs Test Scenario: What's the Difference?

What is a Test Scenario?

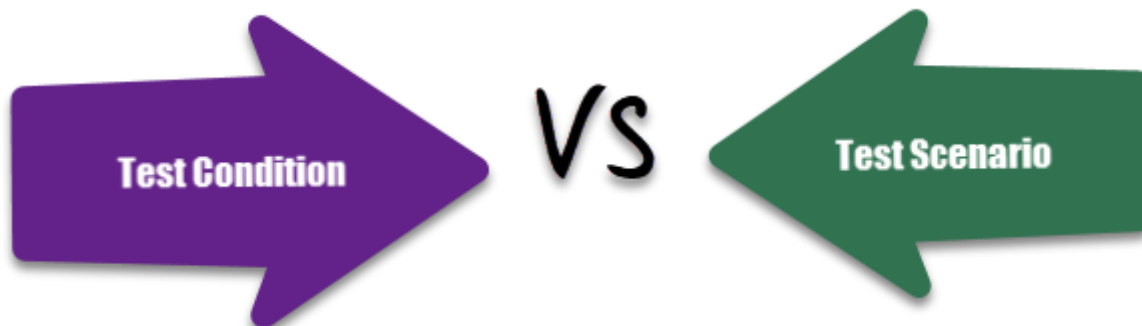
A **Test Scenario** is a probable way or method to test an Application.

What is a Test Condition?

Test Condition is the specification that a tester must follow for testing an Application. There can be multiple Test Conditions in a Test Scenario.

Difference between Test Scenario and Test Condition is a very common FAQ amongst QA beginners.

Below is a detailed comparison



Test Scenario	Test Condition
• The test scenario is a possible way to test an application.	• The test condition is the constraint that you should follow to test an application.
• Test scenario can be a single or a group of test cases	• Test condition can be a piece of functionality or anything you want to verify. In simple terms the goal of a test cases

It is important when the time is less and most team members understand the details from one line scenario	It is an item or event of a system that could be verified by one or more test cases. Eg; transaction, function, structural element etc.
Good Test coverage can be achieved by dividing application in test scenarios which reduces the complexity	Good Test Condition ensure a system is bug-free
The test scenario is rather vague and covers a wide range of possibilities	Test condition are very specific
Example: For testing, you have so many ways like positive testing, negative testing, BVA etc.	Example: When User Name and Password are valid then an application will move forward

Unit Test vs Integration Test: What's the Difference?

What is the Unit Test?

Unit Tests are conducted by developers and test the unit of code(aka module, component) he or she developed. It is a testing method by which individual units of source code are tested to determine if they are ready to use. It helps to reduce the cost of bug fixes since the bugs are identified during the early phases of the development lifecycle.

What is Integration Test?

Integration testing is executed by testers and tests integration between software modules. It is a software testing technique where individual units

of a program are combined and tested as a group. Test stubs and test drivers are used to assist in Integration Testing. Integration test is performed in two way, they are a bottom-up method and the top-down method.

Below is a detailed comparison between the two-

Unit test	Integration test
• The idea behind Unit Testing is to test each part of the program and show that the individual parts are correct.	• The idea behind Integration Testing is to combine modules in the application and test as a group to see that they are working fine
• It is kind of White Box Testing	• It is kind of Black Box Testing
• It can be performed at any time	• It usually carried out after Unit Testing and before System Testing
• Unit Testing tests only the functionality of the units themselves and may not catch integration errors, or other system-wide issues	• Integrating testing may detect errors when modules are integrated to build the overall system
• It starts with the module specification	• It starts with the interface specification
• It pays attention to the behavior of single modules	• It pays attention to integration among modules
• Unit test does not verify whether your code works with external	• Integration tests verify that your code works with external

dependencies correctly.	dependencies correctly.
It is usually executed by the developer	It is usually executed by a test team
Finding errors is easy	Finding errors is difficult
Maintenance of unit test is cheap	Maintenance of integration test is expensive

KEY DIFFERENCE

- Unit testing is a testing method by which individual units of source code are tested to determine if they are ready to use, whereas Integration testing checks integration between software modules.
- Unit Testing test each part of the program and shows that the individual parts are correct, whereas Integration Testing combines different modules in the application and test as a group to see they are working fine.
- Unit Testing starts with the module specification, while Integration Testing starts with interface specification.
- Unit Testing can be performed at any time, on the other hand, Integration Testing is performed after unit testing and before system testing.
- Unit Testing is executed by the developer, whereas Integration Testing is performed by the testing team.
- Unit Testing errors, can be found easily, whereas Integration Testing it is difficult to find errors.
- Unit Testing is a kind of white box testing, whereas Integration Testing is a kind of black-box testing.

Functional Testing Vs Non-Functional Testing: What's the Difference?

What is Functional Testing?

Functional testing is a type of testing which verifies that each **function** of the software application operates in conformance with the requirement specification. This testing mainly involves black box testing, and it is not concerned about the source code of the application.

Every functionality of the system is tested by providing appropriate input, verifying the output and comparing the actual results with the expected results. This testing involves checking of User Interface, APIs, Database, security, client/ server applications and functionality of the Application Under Test. The testing can be done either manually or using automation.

What is Non-Functional Testing?

Non-functional testing is a type of testing to check non-functional aspects (performance, usability, reliability, etc.) of a software application. It is explicitly designed to test the readiness of a system as per nonfunctional parameters which are never addressed by functional testing.

A good example of non-functional test would be to check how many people can simultaneously login into a software.

Non-functional testing is equally important as functional testing and affects client satisfaction.

Functional Vs. Non-Functional Testing



Parameters	Functional	Non-functional testing
------------	------------	------------------------

Parameters	Functional	Non-functional testing
Execution	It is performed before non-functional testing.	It is performed after the functional testing.
Focus area	It is based on customer's requirements.	It focusses on customer's expectation.
Requirement	It is easy to define functional requirements.	It is difficult to define the requirements for non-functional testing.
Usage	Helps to validate the behavior of the application.	Helps to validate the performance of the application.
Objective	Carried out to validate software actions.	It is done to validate the performance of the software.
Requirements	Functional testing is carried out using the functional specification.	This kind of testing is carried out by performance specifications
Manual testing	Functional testing is easy to execute by manual testing.	It's very hard to perform non-functional testing manually.
Functionality	It describes what the product does.	It describes how the product works.
Example Test Case	Check login functionality.	The dashboard should load in 2 seconds.
Testing Types	Examples of Functional Testing Types • Unit testing • Smoke testing • User Acceptance • Integration Testing • Regression testing • Localization • Globalization • Interoperability	Examples of Non-functional Testing Types • Performance Testing • Volume Testing • Scalability • Usability Testing • Load Testing • Stress Testing • Compliance Testing • Portability Testing • Disaster Recover Testing

KEY DIFFERENCE

- Functional testing verifies each function/feature of the software whereas Non Functional testing verifies non-functional aspects like performance, usability, reliability, etc.

- Functional testing can be done manually whereas Non Functional testing is hard to perform manually.
- Functional testing is based on customer's requirements whereas Non Functional testing is based on customer's expectations.
- Functional testing has a goal to validate software actions whereas Non Functional testing has a goal to validate the performance of the software.
- A Functional Testing example is to check the login functionality whereas a Non Functional testing example is to check the dashboard should load in 2 seconds.
- Functional describes what the product does whereas Non Functional describes how the product works.
- Functional testing is performed before the non-functional testing.

Black Box Testing Vs. White Box Testing: Key Differences

What is Black Box testing?

In Black-box testing, a tester doesn't have any information about the internal working of the software system. Black box testing is a high level of testing that focuses on the behavior of the software. It involves testing from an external or end-user perspective. Black box testing can be applied to virtually every level of software testing: unit, integration, system, and acceptance.

What is White Box testing?

White-box testing is a testing technique which checks the internal functioning of the system. In this method, testing is based on coverage of code statements, branches, paths or conditions. White-Box testing is considered as low-level testing. It is also called glass box, transparent box, clear box or code base testing. The white-box Testing method assumes that the path of the logic in a unit or program is known.

Difference between Black Box testing and White Box testing



Parameter	Black Box testing	White Box testing
Definition	It is a testing approach which is used to test the software without the knowledge of the internal structure of program or application.	It is a testing approach in which internal structure is known to the tester.
Alias	It also knowns as data-driven, box testing, data-, and functional testing.	It is also called structural testing, clear box testing, code-based testing, or glass box testing.
Base of Testing	Testing is based on external expectations; internal behavior of the application is unknown.	Internal working is known, and the tester can test accordingly.
Usage	This type of testing is ideal for higher levels of testing like System Testing, Acceptance testing.	Testing is best suited for a lower level of testing like Unit Testing, Integration testing.
Programming knowledge	Programming knowledge is not needed to perform Black Box testing.	Programming knowledge is required to perform White Box testing.
Implementation knowledge	Implementation knowledge is not requiring doing Black Box testing.	Complete understanding needs to implement WhiteBox testing.
Automation	Test and programmer are dependent on each other, so it is tough to automate.	White Box testing is easy to automate.
Objective	The main objective of this testing is	The main objective of

Parameter	Black Box testing	White Box testing
	to check what functionality of the system under test.	White Box testing is done to check the quality of the code.
Basis for test cases	Testing can start after preparing requirement specification document.	Testing can start after preparing for Detail design document.
Tested by	Performed by the end user, developer, and tester.	Usually done by tester and developers.
Granularity	Granularity is low.	Granularity is high.
Testing method	It is based on trial and error method.	Data domain and internal boundaries can be tested.
Time	It is less exhaustive and time-consuming.	Exhaustive and time-consuming method.
Algorithm test	Not the best method for algorithm testing.	Best suited for algorithm testing.
Code Access	Code access is not required for Black Box Testing.	White box testing requires code access. Thereby, the code could be stolen if testing is outsourced.
Benefit	Well suited and efficient for large code segments.	It allows removing the extra lines of code, which can bring in hidden defects.
Skill level	Low skilled testers can test the application with no knowledge of the implementation of programming language or operating system.	Need an expert tester with vast experience to perform white box testing.
Techniques	Equivalence partitioning is Black box testing technique is used for Blackbox testing. Equivalence partitioning divides input values into valid and invalid partitions and selecting corresponding values from each partition of the test data. Boundary value analysis	Statement Coverage, Branch coverage, and Path coverage are White Box testing technique. Statement Coverage validates whether every line of the code is executed at least once. Branch coverage validates

Parameter	Black Box testing	White Box testing
	checks boundaries for input values.	whether each branch is executed at least once Path coverage method tests all the paths of the program.
Drawbacks	Update to automation test script is essential if you to modify application frequently.	Automated test cases can become useless if the code base is rapidly changing.

KEY DIFFERENCE

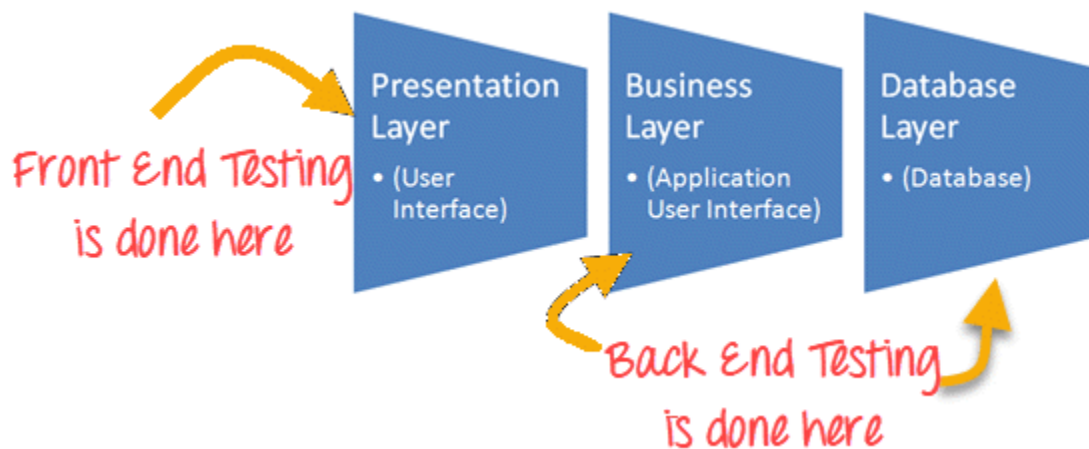
- In Black Box, testing is done without the knowledge of the internal structure of program or application whereas in White Box, testing is done with knowledge of the internal structure of program.
- Black Box test doesn't require programming knowledge whereas the White Box test requires programming knowledge.
- Black Box testing has the main goal to test the behavior of the software whereas White Box testing has the main goal to test the internal operation of the system.
- Black Box testing is focused on external or end-user perspective whereas White Box testing is focused on code structure, conditions, paths and branches.
- Black Box test provides low granularity reports whereas the White Box test provides high granularity reports.
- Black Box testing is a not time-consuming process whereas White Box testing is a time-consuming process.

Frontend Testing Vs. Backend Testing: What's the Difference?

What is Frontend Testing?

Frontend Testing is a type of testing that checks the Presentation layer of a 3 Tier Architecture.

In layman's term, you are checking the GUI - anything that is visible on screen, client -side. For a web application, front-end testing would involve checking functionalities like forms, graphs, menus, reports, etc. as well as associated Javascript. Frontend testing is a term that covers a variety of testing strategies. A tester needs a good understanding of business requirements to perform this type of testing.



What is Back-end Testing?

Backend testing is a type of testing that checks the Application and Database layer of a 3 Tier Architecture.

In a complex software application like ERP, back-end testing would entail checking the business logic in the Application Layer. For simpler applications, backend testing checks the server-side or Database. It means that data entered in the front end will be checked in the back-end database. The database format can be SQL Server, MySQL, Oracle, DB2, etc. The data will be organized in the tables as a record.

Databases are checked for ACID properties, CRUD operations, their Schema, Business rule conformance. Databases is also checked for Security and Performance.

In back-end testing, there is no need to use the GUI. You can directly pass the data using a browser with the parameters required for the function to get the response in some default format. For Example, XML or JSON. You also connect to the database directly and verify the data using SQL queries.

Frontend Testing Vs Backend Testing

Frontend testing	Backend testing
Frontend testing is always performed on the GUI.	Back End Testing involves databases and business logic testing.
The tester must be knowledgeable about the business requirements as well as the usage of the automation frameworks tools.	The tester to be able to perform back-end testing must have a strong background in the database and Structured Query Language (SQL) concepts.
GUI is used to perform the Testing	GUI may or may not be used to perform Testing
It does not need any information to be stored in a database.	It does need information stored in the database.
It is essential to check the overall functionality of the application.	Backend testing is important to check for deadlock, data corruption, data loss, etc
Types of Testing done are – Unit Tests, Acceptance Testing, Accessibility Testing, Regression Testing, etc.	Three widely used types of database testing are SQL Testing, API Testing, etc.

Important Front-end testing tools:

There are many tools available for front-end testing. Here, given are three popular front-end testing tools.

1. Grunt:

Grunt is one of the preferred tools when it comes to task automation. It is a JavaScript task runner, offering plenty of bundled plugins for common tasks.

2. LiveReload:

LiveReload is a simple Web protocol. It triggers events to the clients whenever files are modified. Clients can handle this event in their way, even if the most common use case is when a file is modified.

3. Karma:

Karma is a JavaScript test runner tool. It allows you to execute tests from workstation to the production CI.

Important Backend testing tools:

Database testing also refers to Back-end testing is very important. Here, are some important back-end testing tools which help to find issues like deadlocking, data corruption, and poor performance.

1. Data Factory:

Data factory is database testing tool. It works as data generator and data manager for database testing. It has very easy to use interface and capable of managing complicated data relationship.

2. Data Generator:

DTM Data Generator is another backend testing tool. It is used for generating data rows and schema objects for database testing. The tool supports Load Usability and performance testing on the database.

3. TurboData

Turbodata software tool can be used to generate test data with foreign keys. It allows to use Select, Updates, and Delete SQL commands. It also supports multiple sequential files and relational databases.

KEY DIFFERENCE

- Frontend Testing checks the presentation layer of a 3 Tier Architecture whereas backend testing checks the application and database layer of a 3 Tier Architecture.
- Frontend testing is always performed on the GUI whereas backend Testing involves databases and business logic testing.
- Frontend testing does not need any information to be stored in a database, but backend testing needs information stored in the database.
- Frontend testing is essential to check the overall functionality of the application while backend testing is important to check for deadlock, data corruption, data loss, etc.
- Frontend tester must be knowledgeable about the business requirements and automation frameworks tools while Backend tester must have a strong background in the database and Structured Query Language (SQL) concepts.
- Frontend testing examples are Unit Tests, Acceptance Testing, Accessibility Testing, Regression Testing whereas backend testing examples are SQL Testing, API Testing, etc.

Difference Between Retesting and Regression Testing

What is Re-testing?

Re-testing is a type of testing performed to check the test cases that were unsuccessful in the final execution are successfully pass after the defects are repaired.

What is Regression Testing?

Regression Testing is a type of software testing executed to check whether a code change has not unfavorably disturbed current features & functions of an Application.

Re-testing Vs Regression Testing is a common FAQ amongst QA aspirants.

Below is a detailed comparison with Example

Retesting vs Regression Testing

Regression Testing	Re-testing
Regression Testing is carried out to confirm whether a recent program or code change has not adversely affected existing features	Re-testing is carried out to confirm the test cases that failed in the final execution are passing after the defects are fixed
The purpose of Regression Testing is that new code changes should not have any side effects to existing functionalities	Re-testing is done on the basis of the Defect fixes
Defect verification is not the part of Regression Testing	Defect verification is the part of re-testing
Based on the project and availability of resources, Regression Testing can be carried out parallel with Re-testing	Priority of re-testing is higher than regression testing, so it is carried out before regression testing
You can do automation for regression testing, Manual Testing could be	You cannot automate the test cases for Retesting

expensive and time-consuming	
Regression testing is known as a generic testing	Re-testing is a planned testing
Regression testing is done for passed test cases	Retesting is done only for failed test cases
Regression testing checks for unexpected side-effects	Re-testing makes sure that the original fault has been corrected
Regression testing is only done when there is any modification or changes become mandatory in an existing project	Re-testing executes a defect with the same data and the same environment with different inputs with a new build
Test cases for regression testing can be obtained from the functional specification, user tutorials and manuals, and defect reports in regards to corrected problems	Test cases for retesting cannot be obtained before start testing.

KEY DIFFERENCE

- Regression testing is performed for passed test cases while Retesting is done only for failed test cases.
- Regression testing checks for unexpected side-effects while Re-testing makes sure that the original fault has been corrected.
- Regression Testing doesn't include defect verification whereas Re-testing includes defect verification.
- Regression testing is known as generic testing whereas Re-testing is planned testing.

- Regression Testing is possible with the use of automation whereas Re-testing is not possible with automation.

Static Testing vs Dynamic Testing: What's the Difference?

What is Static Testing?

Under **Static Testing**, code is not executed. Rather it manually checks the code, requirement documents, and design documents to find errors. Hence, the name "static".

The main objective of this testing is to improve the quality of software products by finding errors in the early stages of the development cycle. This testing is also called a Non-execution technique or verification testing.

Static testing involves manual or automated reviews of the documents. This review is done during an initial phase of testing to catch [Defect](#) early in STLC. It examines work documents and provides review comments

Examples of Work documents-

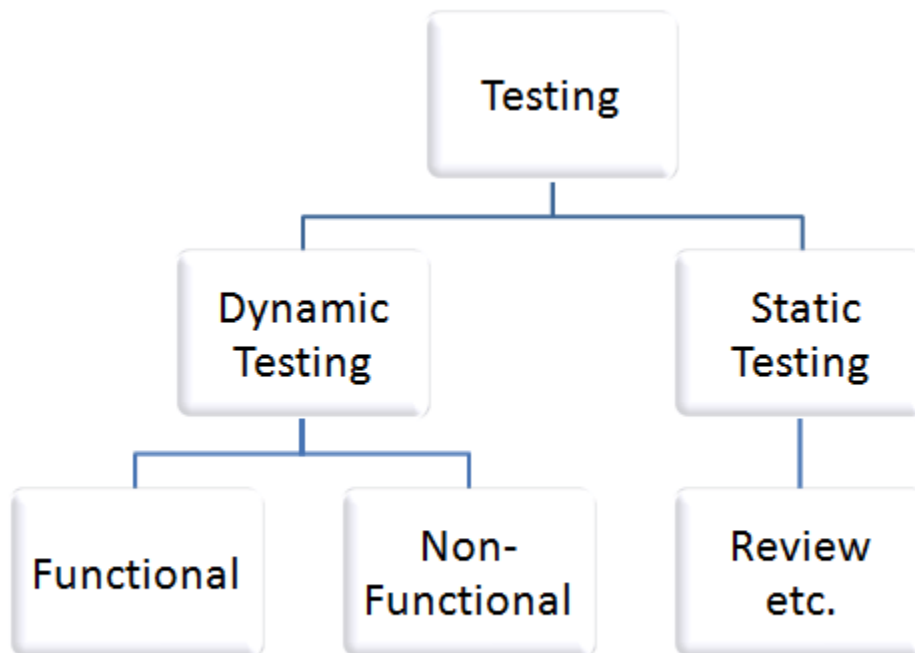
- Requirement specifications
- Design document
- Source Code
- Test Plans
- Test Cases
- Test Scripts
- Help or User document
- Web Page content

What is Dynamic Testing?

Under **Dynamic Testing**, a code is executed. It checks for functional behavior of software system, memory/cpu usage and overall performance of the system. Hence the name "Dynamic"

The main objective of this testing is to confirm that the software product works in conformance with the business requirements. This testing is also called an Execution technique or validation testing.

Dynamic testing executes the software and validates the output with the expected outcome. Dynamic testing is performed at all levels of testing and it can be either black or white box testing.



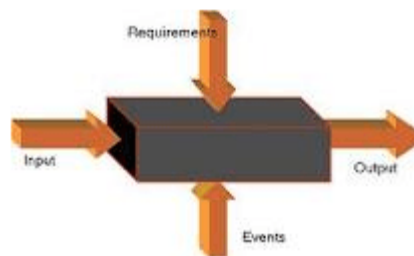
@guru99.com

Static Testing Techniques:

- **Informal Reviews:** This is one of the type of review which doesn't follow any process to find errors in the document. Under this technique, you just review the document and give informal comments on it.
- **Technical Reviews:** A team consisting of your peers, review the technical specification of the software product and checks whether it is suitable for the project. They try to find any discrepancies in the specifications and standards followed. This review concentrates mainly on the technical documentation related to the software such as Test Strategy, [Test Plan](#) and requirement specification documents.

- **Walkthrough:** The author of the work product explains the product to his team. Participants can ask questions if any. A meeting is led by the author. Scribe makes note of review comments
- **Inspection:** The main purpose is to find defects and meeting is led by a trained moderator. This review is a formal type of review where it follows a strict process to find the defects. Reviewers have a checklist to review the work products. They record the defect and inform the participants to rectify those errors.
- **Static code Review:** This is a systematic review of the software source code without executing the code. It checks the syntax of the code, coding standards, code optimization, etc. This is also termed as white box testing. This review can be done at any point during development.

Dynamic Testing Techniques:



- **Unit Testing:** Under Unit Testing, individual units or modules are tested by the developers. It involves testing of source code by developers.
- **Integration Testing:** Individual modules are grouped together and tested by the developers. The purpose is to determine what modules are working as expected once they are integrated.
- **System Testing:** [System Testing](#) is performed on the whole system by checking whether the system or application meets the requirement specification document.

Also, Non-functional testing like performance, [Security Testing](#) fall under the category of dynamic testing.

Difference between Static and Dynamic Testing:



Static Testing	Dynamic Testing
Testing was done without executing the program	Testing is done by executing the program
This testing does the verification process	Dynamic testing does the validation process
Static testing is about prevention of defects	Dynamic testing is about finding and fixing the defects
Static testing gives an assessment of code and documentation	Dynamic testing gives bugs/bottlenecks in the software system.
Static testing involves a checklist and process to be followed	Dynamic testing involves test cases for execution
This testing can be performed before compilation	Dynamic testing is performed after compilation
Static testing covers the structural and statement coverage testing	Dynamic testing techniques are Boundary Value Analysis & Equivalence Partitioning.
Cost of finding defects and fixing is less	Cost of finding and fixing defects is high
Return on investment will be high as this process involved at an early stage	Return on investment will be low as this process involves after the development phase
More reviews comments are highly	More defects are highly recommended

recommended for good quality	for good quality.
Requires loads of meetings	Comparatively requires lesser meetings

KEY DIFFERENCE

- Static testing was done without executing the program whereas Dynamic testing is done by executing the program.
- Static testing checks the code, requirement documents, and design documents to find errors whereas Dynamic testing checks the functional behavior of software system, memory/CPU usage and overall performance of the system.
- Static testing is about the prevention of defects whereas Dynamic testing is about finding and fixing the defects.
- Static testing does the verification process while Dynamic testing does the validation process.
- Static testing is performed before compilation whereas Dynamic testing is performed after compilation.
- Static testing techniques are structural and statement coverage while Dynamic testing techniques are Boundary Value Analysis & Equivalence Partitioning.

Test Strategy vs Test Plan: What's the Difference?

What is Test Plan?

A **Test Plan** is defined as a document which outlines the scope, objective, method and weight on a software testing task.

What is Test Strategy?

Test Strategy is defined as a set of guiding principle that enlightens test design & regulates how testing needs to be done.

Test Plan V/s Test Strategy is a prominent confusion among multiple levels of QA Aspirants

Below is the detailed guide to it

Difference Between Test Strategy and Test Plan

Test Plan	Test Strategy
• A test plan for software project can be defined as a document that defines the scope, objective, approach and emphasis on a software testing effort	• Test strategy is a set of guidelines that explains test design and determines how testing needs to be done
• Components of Test plan include- Test plan id, features to be tested, test techniques, testing tasks, features pass or fail criteria, test deliverables, responsibilities, and schedule, etc.	• Components of Test strategy includes- objectives and scope, documentation formats, test processes, team reporting structure, client communication strategy, etc.
• Test plan is carried out by a testing manager or lead that describes how to test, when to test, who will test and what to test	• A test strategy is carried out by the project manager. It says what type of technique to follow and which module to test
• Test plan narrates about the specification	• Test strategy narrates about the general approaches
• Test plan can change	• Test strategy cannot be changed
• Test planning is done to determine possible issues and dependencies in order to identify the risks.	• It is a long-term plan of action. You can abstract information that is not project specific and put it into test approach

• A test plan exists individually	• In smaller project, test strategy is often found as a section of a test plan
• It is defined at project level	• It is set at organization level and can be used by multiple projects

KEY DIFFERENCE

- Test Plan is a document that describes the scope, objective and weight on software testing task whereas Test Strategy describes how testing needs to be done.
- Test Plan is used at the project level whereas Test Strategy is used at the organization level.
- Test Plan has the primary goal of how to test, when to test and who will verify whereas Test Strategy has the primary goal of what technique to follow and which module to check.
- Test Plan can be changed whereas Test Strategy can't change.
- Test Plan is carried out by the test manager whereas the Test Strategy is carried out by the project manager.